

PTA Engenharia de Dados

“Dados brutos são apenas números; dados processados são inteligência.”

Sobre o projeto (Visão de negócio)

A maioria das empresas guarda seus dados em planilhas ou arquivos avulsos, o que dificulta extrair informações úteis sem depender de atividades manuais e repetitivas. Este projeto foi criado para resolver esse problema, automatizando todo o processo: desde a coleta de dados desorganizados até a transformação deles em conhecimento pronto para decisões. Assim, o risco de erros e retrabalho diminui, e o tempo do cliente é preservado para o que realmente importa: analisar e agir.

Como funciona (Visão intuitiva)

Imagine uma linha de montagem inteligente para dados:

- O pipeline é inicializado num fluxo ativo de 30 em 30 minutos analisando e recebendo informações de dados novos na planilha.
- Um sistema-orquestrador (n8n) percebe esse movimento e inicia a solução.
- Os dados são enviados automaticamente para um motor de processamento (FastAPI), que:
 - o Valida (confere se estão no formato certo, igual a um porteiro checando os convites).
 - o Limpa (remove informações irrelevantes, como ruídos ou espaços em branco).
 - o Aplica regras de negócio (adapta os dados conforme as necessidades).
- Só então os dados tratados vão para o lugar seguro onde ficam disponíveis para análises e relatórios.

E de forma mais técnica, a arquitetura ETL funciona da seguinte forma:

Extração (Extract): O n8n monitora planilhas do Google Sheets (novas vendas, novos produtos, etc).

Transformação (Transform): Os dados brutos são enviados para uma API desenvolvida em FastAPI (Python). Lá, eles são limpos, validados e padronizados.

Carga (Load): Os dados tratados retornam e são salvos no Google Sheets prontos para análise, e um e-mail de confirmação é enviado.

Esse fluxo reduz drasticamente a chance de erro humano e garante que apenas dados corretos cheguem ao fim do caminho.

Orquestração no n8n

O n8n atua como o maestro do fluxo. Abaixo, detalhamos como cada workflow foi estruturado com base nos diagramas do projeto.

Lógica Geral (Carga Incremental)

Todos os fluxos utilizam uma lógica de carga incremental para evitar processamento duplicado:

1. O fluxo lê uma planilha de configuração (ler config) para descobrir qual foi a última linha processada.
2. Lê os dados novos a partir dessa linha.
3. Após o sucesso, atualiza a configuração com o número da nova última linha (Update ultima_linha).

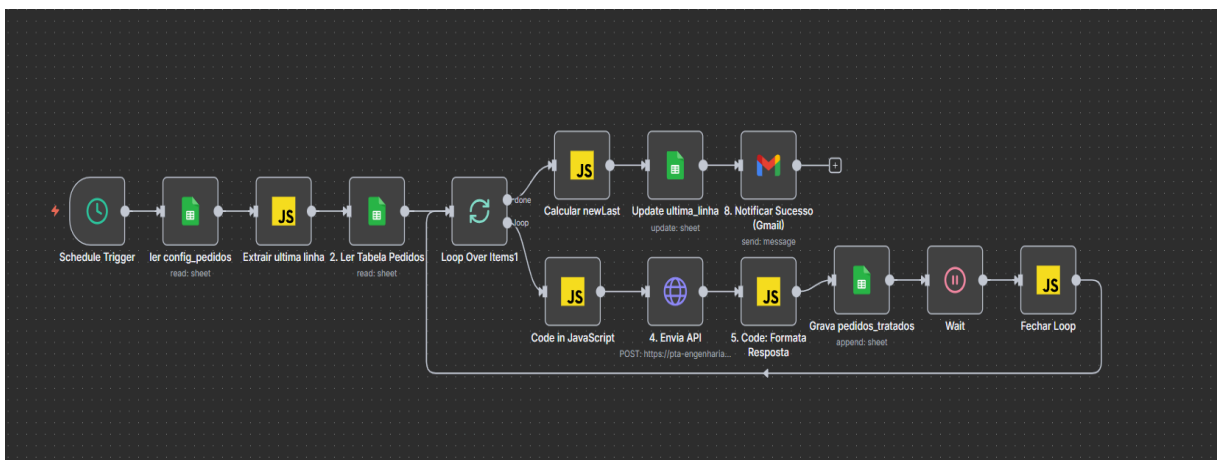
Detalhamento dos Workflows

Fluxo de Pedidos (fluxo_pedidosN8N) e Itens (fluxo_itens_pedidosN8N)

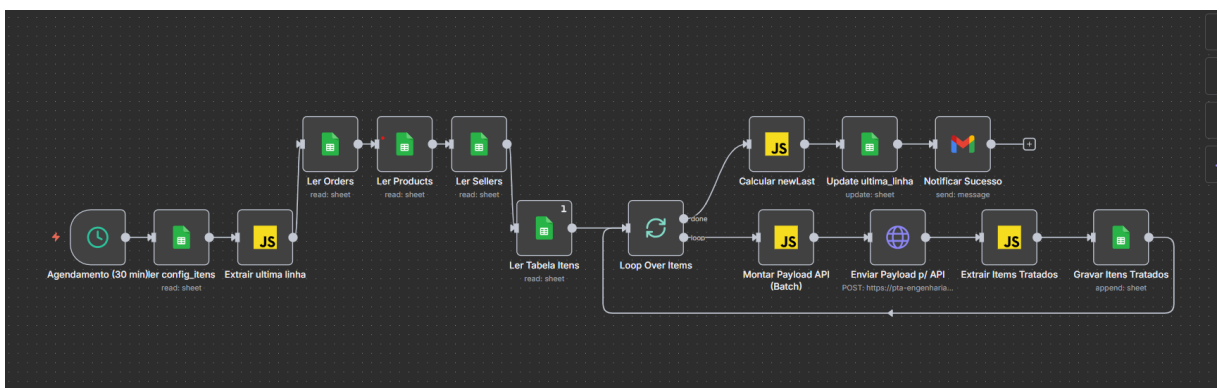
Estes fluxos lidam com alto volume e complexidade, utilizando Loops e Batch Processing.

- Trigger: Agendamento temporal (Schedule Trigger).
- Extração: Lê a tabela de origem respeitando a última linha processada.

- **Processamento em Lote (Loop):**
 - Para evitar timeout da API, os dados são processados em pequenos pacotes dentro do nó Loop Over Items.
 - Envia API: Faz um POST para o endpoint de tratamento.
 - Wait: Existe uma pausa estratégica para não sobrecarregar o servidor.
- **Carga:** Os dados tratados são gravados na planilha de destino.
- **Notificação:** Ao final do loop, o sistema calcula a nova última linha e notifica o sucesso via Gmail.



Esta imagem se trata do fluxo de “pedidos”.



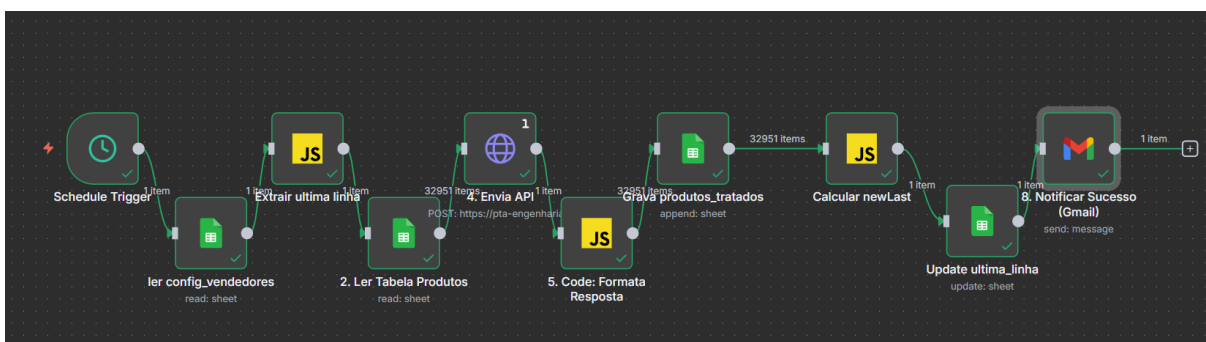
Esta imagem se trata do fluxo de “itens pedidos”.

Fluxo de Vendedores (fluxo_vendedoresN8N) e Produtos (fluxo_productsN8N)

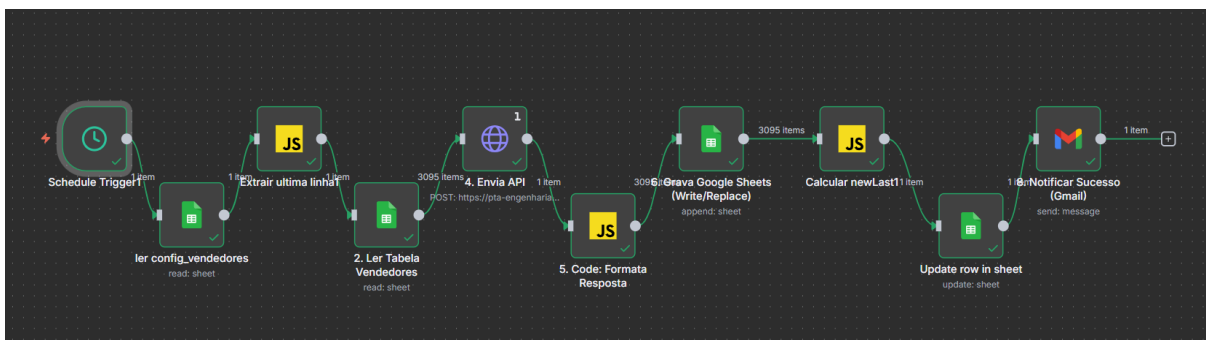
Fluxos otimizados para cargas diretas (Bulk).

- **Extração:** Lê todos os novos cadastros de uma vez.

- API: Envia o JSON completo para a API (nós visualizados processando ~3000 itens de uma vez).
- Code Formata Resposta: Um script JS prepara o JSON retornado pela API para o formato de linhas do Google Sheets.
- Append: Adiciona os dados limpos ao final da planilha de destino.



Esta imagem se trata do fluxo de “produtos”.



Esta imagem se trata do fluxo de “vendedores”.

Endpoints da API e Regras de Limpeza

A API, construída com FastAPI, é o cérebro da operação. Ela expõe a rota `/process/{dataset}`.

Endpoints

- POST `/process`: Recebe dados brutos de todos os DataFrames, separando os tratamentos, respectivamente para cada categoria, dentro da própria API.

Regras de Negócio e Tratamento

Dataset	Principais Tratamentos Realizados
Orders	1. Padronização de datas para formato timestamp (YYYY-MM-DD). 2. Tradução de status do pedido. 3. Criação de novas colunas derivadas: tempo_entrega_dias; tempo_entrega_estimado_dias; diferença_entrega_dias; entrega_no_prazo; erro_cronológico.
Order Items	1. Conversão de tipos numéricos (preço, frete). 2. Tratamento de nulos (mediana). 3. Verificação de integridade referencial (Order ID).
Products	1. Padronização do texto. valores lower_case; espacos() por underscore(_); valores nulos por "indefinido". 2. Tratamento de campos nulos(mediana).
Sellers	1. Formatação de Cidade/Estado:

Tecnologias utilizadas

Ferramenta	Para que serve na solução?
Python	Linguagem principal usada para criar o motor de processamento de dados. Permite alto grau de automação e leitura fácil de código.
FastAPI	Framework em Python para construir APIs (pontes de comunicação). Ele recebe dados, aplica as regras e expõe resultados para serem consumidos por outros sistemas. ^{[1][2][3][4]}
n8n	Ferramenta visual de automação que age como maestro do fluxo: conecta diferentes sistemas, lida com gatilhos (por exemplo, chegada de novos dados), e encaminha tudo automaticamente. ^{[5][6][7][8][9][10]}
Docker	Garante que a solução funcione igualzinha em qualquer computador, sem problemas de ambiente. Cada parte da solução roda em seu próprio “recipiente”, isolado e padronizado.
GitHub	Repositório online para guardar o código-fonte e documentar as etapas, garantindo histórico e segurança para evoluções futuras.

Guia de instalação e uso

Antes de começar

É importante garantir que o seu computador possui o básico:

- Python 3.10+ – responsável por rodar o motor lógico.
- Git – ferramenta para baixar o projeto facilmente.

Opção 1: Rodar localmente (Desenvolvimento)

1. Baixe o projeto:

- o Abra o terminal (Windows ou Mac/Linux) e digite:

```
git clone https://github.com/mariasoaresh/PTA-engenharia-de-dados.git
cd PTA-engenharia-de-dados
```

Por que? Assim você traz todos os arquivos necessários para sua máquina.

2. Prepare o ambiente:

- o Crie um ambiente virtual (evita "misturar" as bibliotecas do projeto com outras do seu computador):

```
python -m venv venv
```

- o Ative o ambiente:

- Windows: `venv\Scripts\activate`
- Linux/Mac: `source venv/bin/activate`

3. Instale as dependências:

```
pip install -r./ requirements.txt
```

Por que? Assim todo o conjunto de ferramentas usado pelo projeto é instalado automaticamente.

4. Configure as variáveis de ambiente:

- o Crie um arquivo chamado `.env` (pode copiar o modelo `.env.example` que veio no projeto) e preencha as informações solicitadas, como senhas ou chaves.

5. **Execute o servidor do projeto:**

```
uvicorn app.main:app --reload
```

Por que? Assim o motor de processamento de dados entra em funcionamento, pronto para receber informações via web.

6. **Acesse a documentação interativa (opcional):**

- o No navegador, vá para <http://localhost:8000/docs>
- o Assim é possível testar manualmente a solução, ver exemplos e entender o que pode ser feito.

Opção 2: Rodar com docker (Recomendado para produção)

1. **Certifique-se de ter Docker instalado.**

- o <https://www.docker.com/get-started>

2. **No terminal, execute:**

```
docker-compose up --build
```

Por que? O Docker monta “mini-computadores virtuais”, garantindo que tudo funciona do mesmo jeito em qualquer máquina.

3. **Pronto!**

- o A API do projeto deve estar disponível em <http://localhost:8000>

Contato e suporte

Se tiver dúvidas ou dificuldade com qualquer etapa, entre em contato com a equipe:

- GitHub do projeto (issues):
<https://github.com/mariasoaresh/PTA-engenharia-de-dados/issues>
- Link da API pública:
<https://pta-engenharia-de-dados-1-ywa7.onrender.com/>

- **Equipe CITi – UFPE**

Maria Eduarda Soares

Theo Barza

Vinicius Leôncio

Esdras Vinicius